



FUTURE INTERNS

API Security Assessment Report

Cyber Security Internship – Task 3 (2026)

Target: JSONPlaceholder Public API

prepared By: Yasaswini Vanukuri

Assessment Type: Read-Only Security Analysis

Executive Summary

This report presents the findings of a security assessment conducted on the publicly available JSONPlaceholder API.

The objective of this assessment was to analyze potential API security risks aligned with the OWASP API Security Top 10 framework.

The assessment was performed using safe, read-only techniques without exploiting or modifying any data.

Key observations include:

- Unauthenticated access to user data
- Excessive exposure of user details
- Lack of object-level authorization controls
- Limited rate-limiting enforcement
- Proper input validation and error handling

Overall, while the API is designed for demonstration purposes, similar implementations in production systems could lead to significant security risks.

Scope of Assessment

Scope

- Public demo API: JSONPlaceholder
- Read-only testing
- No exploitation performed

Tools Used

- Kali Linux (curl commands)
- Browser DevTools
- MS Word and GitHub

Methodology

The assessment included:

1. Endpoint discovery
2. Response analysis
3. Authentication verification
4. Authorization testing (IDOR)
5. Rate limiting evaluation
6. Input validation testing

Detailed Findings

Observation

The /users endpoint is accessible without any authentication mechanism.

Example:

```
curl https://jsonplaceholder.typicode.com/users
```

Risk: In production systems, lack of authentication may allow unauthorized users to access sensitive data.

Severity: Medium

Finding 2: Excessive Data Exposure

Observation

The API returns complete user profiles including:

- Name
- Email
- Phone
- Address
- Geo-location coordinates
- Company details

Risk: Exposing unnecessary fields increases attack surface and potential for misuse.

Severity: High

Finding 3: Broken Object Level Authorization (IDOR)

Observation

Changing user ID in the URL returns data of other users.

Example:

```
curl https://jsonplaceholder.typicode.com/users/1
```

```
curl https://jsonplaceholder.typicode.com/users/2
```

Risk: In real-world applications, this could allow attackers to access other users' data without authorization checks.

Severity: High

Finding 4: Rate Limiting Assessment

Multiple repeated requests were sent in a loop.

Observation:

- No immediate blocking
- Rate limit headers present
- High threshold before restriction

Risk: High rate limits may allow automated scraping or abuse in production systems.

Severity: Low–Medium

Finding 5: Input Validation & Error Handling

Invalid inputs tested:

```
curl https://jsonplaceholder.typicode.com/users/abc
```

```
curl https://jsonplaceholder.typicode.com/users/9999999
```

Response: {}

Observation

The API gracefully handles invalid inputs without exposing stack traces or internal errors.

Risk: No major risk observed. Proper secure error handling implemented.

Severity: Informational (Good Practice)

Risk Summary

Vulnerability	Severity
Unauthenticated Access	Medium
Excessive Data Exposure	High
Broken Object Level Authorization	High
Rate Limiting	Low–Medium
Input Validation	Informational

Recommendations

- Implement authentication (JWT/API Keys)
- Apply strict authorization checks
- Minimize exposed response fields
- Enforce strict rate limiting
- Follow OWASP API Security Top 10 guidelines

Overall Risk Assessment

If deployed in a production SaaS environment, the identified issues could lead to data exposure, regulatory violations, and reputational damage. Strong authentication and authorization controls are recommended before production deployment.

Conclusion

The API demonstrates typical vulnerabilities that may appear in poorly secured production APIs, particularly:

- Lack of authentication
- Insufficient authorization controls
- Excessive data exposure

While this API is intentionally designed as a public demo service, the analysis highlights important security considerations developers must implement in real-world systems.

This assessment successfully demonstrates practical understanding of API security fundamentals aligned with OWASP standards.